

6.3 · scikit-learn, métricas y validación

Una guía práctica y completa para dominar la API de scikit-learn, construir pipelines robustos, evaluar modelos con las métricas correctas y optimizar hiperparámetros con rigor científico.

MACHINE LEARNING

SCIKIT-LEARN

PYTHON



Índice de contenidos

Este módulo cubre el ciclo completo de construcción y evaluación de modelos en scikit-learn, desde la API base hasta la selección estratégica de métricas según el contexto de negocio.

01

API de scikit-learn

Estimadores, transformadores y predictores

02

Pipelines y ColumnTransformer

Flujos de trabajo reproducibles

03

Validación cruzada

Evaluación fiable y sin sesgo

04

Métricas de regresión y clasificación

MAE, RMSE, R^2 , precisión, recall, F1

05

Sesgo, varianza y optimización

Curvas de validación y GridSearchCV

CAPÍTULO 1

La API de scikit-learn: estimadores

Todo en scikit-learn gira en torno a una interfaz unificada y elegante. Comprender la API base es el punto de partida imprescindible para trabajar con cualquier algoritmo de la librería.



El patrón Estimator: tres métodos clave

Todos los objetos de scikit-learn comparten la misma interfaz. Independientemente del algoritmo, el flujo siempre es el mismo: instanciar, ajustar y predecir o transformar.



Instanciar

```
modelo =  
RandomForestClassifier(n_estimator  
s=100)
```



Ajustar

```
modelo.fit(X_train, y_train)
```



Predecir

```
modelo.predict(X_test)
```

- ❏ Los **transformadores** añaden el método `transform()`, mientras que los **predictores** implementan `predict()`. Un Pipeline combina ambos de forma secuencial.

Tipos de estimadores en scikit-learn

La librería organiza sus objetos en tres grandes familias según su rol dentro del flujo de trabajo. Conocer esta taxonomía facilita navegar la documentación y combinar componentes correctamente.



Transformadores

Modifican los datos de entrada. Implementan `fit()` y `transform()`.

- StandardScaler
- OneHotEncoder
- PCA
- SimpleImputer



Predictores

Aprenden de los datos y generan predicciones. Implementan `fit()` y `predict()`.

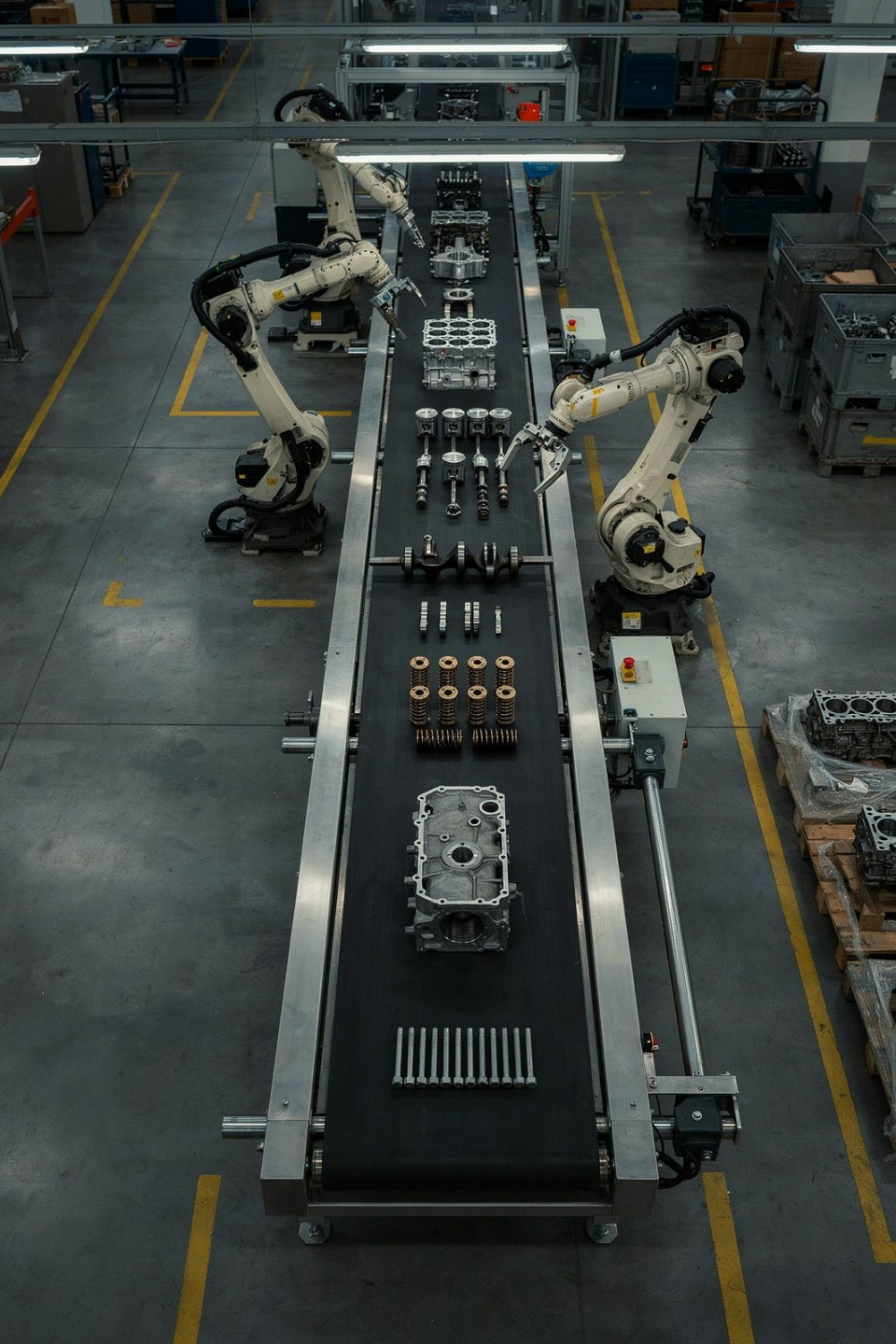
- LinearRegression
- RandomForestClassifier
- SVC
- KNeighborsRegressor



Meta-estimadores

Envuelven a otros estimadores para añadir funcionalidad adicional.

- Pipeline
- GridSearchCV
- BaggingClassifier
- AdaBoostClassifier



CAPÍTULO 2

Pipelines y ColumnTransformer

Los Pipelines encadenan transformaciones y modelos en un único objeto, garantizando que el preprocesamiento sea coherente entre entrenamiento y predicción, y eliminando el riesgo de data leakage.

¿Por qué usar Pipeline?

✗ Sin Pipeline (peligroso)

El preprocesamiento se aplica manualmente en cada paso. Es fácil cometer errores como escalar con estadísticas del test set, provocando **data leakage** y métricas optimistas.

- Código repetitivo y frágil
- Riesgo de contaminación train/test
- Difícil de mantener en producción

✓ Con Pipeline (correcto)

Todo el flujo —imputación, escalado, codificación y modelo— se encapsula en un objeto. El Pipeline aprende únicamente de los datos de entrenamiento y aplica las mismas transformaciones al test set.

- Código limpio y reproducible
- Compatible con cross-validation
- Listo para desplegar en producción

Construcción de un Pipeline básico

Un Pipeline se define como una lista de tuplas (nombre, objeto). El último paso debe ser un predictor; los anteriores deben ser transformadores. scikit-learn ejecuta cada paso en orden.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
```

```
pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(max_iter=500))
])
```

```
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
```

Con este enfoque, `pipe.fit()` llama a `fit_transform()` en cada transformador y finaliza con `fit()` en el modelo. En predicción, `pipe.predict()` aplica `transform()` a cada paso previo.

ColumnTransformer: distintas transformaciones por columna

En datos tabulares reales, las columnas numéricas y categóricas requieren preprocesamiento diferente. `ColumnTransformer` permite aplicar transformaciones distintas a subconjuntos de columnas y combinarlas automáticamente en una sola matriz de salida.



Preprocesado

`ColumnTransformer`: numéricos y categóricos

Matriz de características

Salida combinada lista para modelado

Modelo

Regresión logística para clasificación

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
```

```
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), ['edad', 'salario']),
    ('cat', OneHotEncoder(), ['ciudad', 'sector'])
])
```

```
pipe_full = Pipeline([
    ('prep', preprocessor),
    ('model', LogisticRegression())
])
```

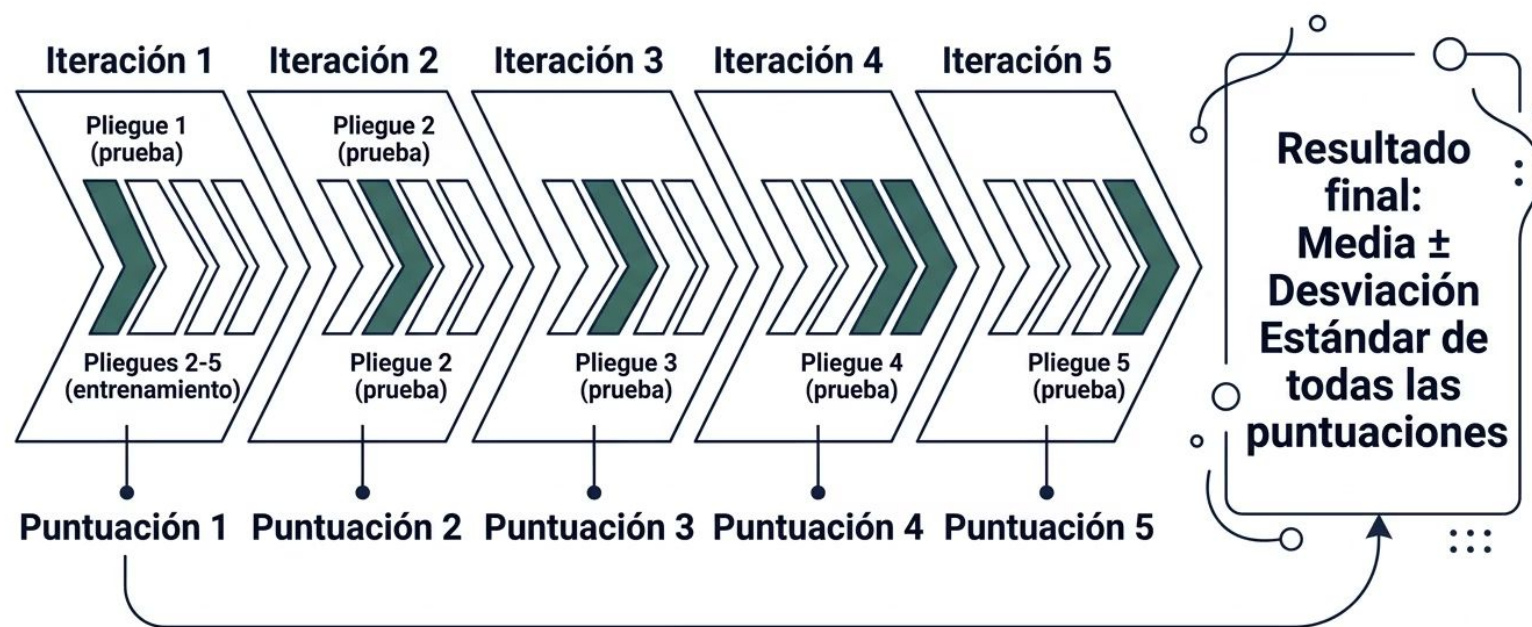
Validación cruzada

Evaluar un modelo sobre los mismos datos usados para entrenarlo produce estimaciones optimistas e irreales. La validación cruzada ofrece una estimación robusta del rendimiento generalizable a datos no vistos.



K-Fold Cross-Validation paso a paso

El conjunto de entrenamiento se divide en **k partes iguales (folds)**. El modelo se entrena k veces: cada iteración usa k-1 folds para entrenar y 1 fold para validar. El rendimiento final es la media de las k puntuaciones.



Un valor habitual es **k=5 o k=10**. Con k más alto se reduce el sesgo pero aumenta el coste computacional. La desviación estándar de las puntuaciones indica la estabilidad del modelo.

cross_val_score y StratifiedKFold

scikit-learn implementa la validación cruzada en una sola línea. Para clasificación desequilibrada, `StratifiedKFold` garantiza que cada fold mantenga la proporción de clases del conjunto original.

```
from sklearn.model_selection import cross_val_score, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_val_score(
    estimator = pipe_full,
    X = X_train,
    y = y_train,
    cv = cv,
    scoring = 'f1_weighted'
)

print(f"F1 medio: {scores.mean():.3f} ± {scores.std():.3f}")
```

 Usa siempre `cross_val_score` sobre el **Pipeline completo**, no sobre los datos ya transformados. De lo contrario, el escalado del fold de validación queda contaminado por los datos de entrenamiento.

Métricas de regresión: MAE, RMSE, R^2

Evaluar cuánto se equivoca un modelo de regresión requiere métricas que cuantifiquen el error en las unidades del problema. Cada métrica tiene fortalezas distintas según el contexto.



Comprendiendo MAE, RMSE y R²

MAE — Error Absoluto Medio

Promedio de los errores absolutos.

Robusto ante outliers porque no eleva al cuadrado. Interpretable: "en promedio, el modelo se equivoca en X unidades". Ideal cuando los errores grandes no son desproporcionadamente más graves.

RMSE — Raíz del Error Cuadrático Medio

Eleva al cuadrado los errores antes de promediar, **penalizando más los errores grandes**. Muy usado en competiciones de ML. Sensible a outliers. Sus unidades son las mismas que la variable objetivo.

R² — Coeficiente de determinación

Mide la **proporción de varianza explicada** por el modelo (0 a 1). Un R²=0.85 significa que el modelo explica el 85% de la variabilidad de y. Puede ser negativo si el modelo es peor que predecir la media.

```
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score
```

```
mae = mean_absolute_error(y_test, y_pred)
```

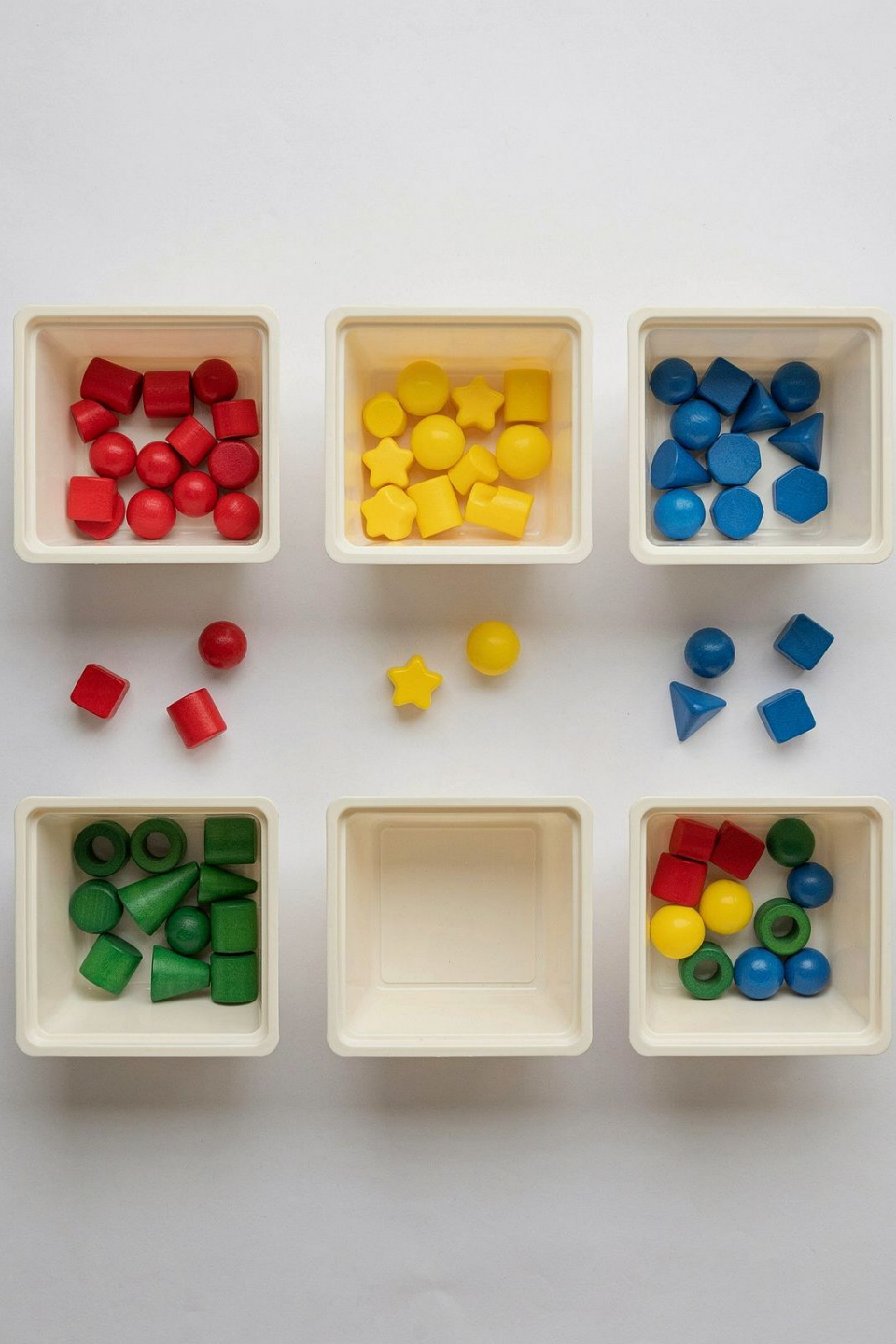
```
rmse = root_mean_squared_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print(f"MAE={mae:.2f} RMSE={rmse:.2f} R2={r2:.3f}")
```

Métricas de clasificación: precisión, recall, F1

En clasificación, la exactitud (accuracy) puede ser engañosa cuando las clases están desequilibradas. Precisión, recall y F1 ofrecen una visión más completa del comportamiento del clasificador.



Precisión, Recall y F1-Score

Precisión (Precision)

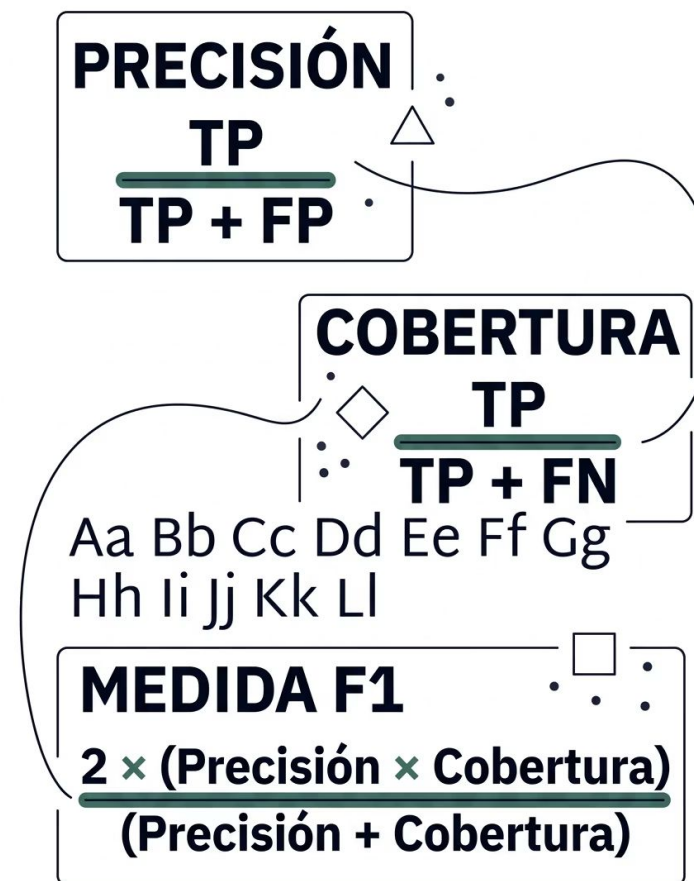
De todas las predicciones positivas, ¿cuántas son realmente positivas? Clave cuando los **falsos positivos son costosos** (ej. spam, diagnósticos invasivos).

Recall (Sensibilidad)

De todos los positivos reales, ¿cuántos detectamos? Crítico cuando los **falsos negativos son peligrosos** (ej. detección de enfermedades graves, fraude financiero).

F1-Score

Media armónica de precisión y recall. Equilibra ambas métricas. Ideal cuando se necesita un **compromiso único** entre las dos.



Calculando métricas de clasificación en scikit-learn

La función `classification_report` ofrece en una sola llamada precisión, recall, F1 y soporte para cada clase, lo que la convierte en el punto de partida ideal para cualquier evaluación de clasificación.

```
from sklearn.metrics import classification_report, f1_score

# Reporte completo por clase
print(classification_report(y_test, y_pred,
                           target_names=['Clase 0', 'Clase 1']))

# F1 promediado (weighted tiene en cuenta el soporte de cada clase)
f1_macro = f1_score(y_test, y_pred, average='macro')
f1_weighted = f1_score(y_test, y_pred, average='weighted')
```

macro

Media no ponderada de todas las clases. Trata igual clases mayoritarias y minoritarias.

weighted

Media ponderada por el soporte (número de muestras). Recomendado con clases desequilibradas.

binary

Sólo para clasificación binaria. Evalúa únicamente la clase positiva.

Matriz de confusión y ROC-AUC

La matriz de confusión y la curva ROC son herramientas visuales imprescindibles para entender en profundidad cómo falla un clasificador y comparar modelos independientemente del umbral de decisión.

Matriz de confusión: anatomía de los errores

¿Qué nos dice la matriz?

Desglosa las predicciones en cuatro categorías para clasificación binaria:

TP (Verdadero Positivo)

Predijo positivo, era positivo ✓

TN (Verdadero Negativo)

Predijo negativo, era negativo ✓

FP (Falso Positivo)

Predijo positivo, era negativo ✗ (Error Tipo I)

FN (Falso Negativo)

Predijo negativo, era positivo ✗ (Error Tipo II)

Código en scikit-learn

```
from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay
)

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['Negativo', 'Positivo']
)
disp.plot(cmap='Blues')
```

Visualizar la matriz como mapa de calor permite detectar patrones de error sistemáticos, como confundir sistemáticamente una clase específica.

ROC-AUC: rendimiento independiente del umbral

La curva ROC representa la tasa de verdaderos positivos (TPR) frente a la tasa de falsos positivos (FPR) para todos los umbrales posibles. El **AUC (Área Bajo la Curva)** resume el rendimiento en un único número entre 0.5 (azar) y 1.0 (perfecto).

```
from sklearn.metrics import roc_auc_score, roc_curve
import matplotlib.pyplot as plt
```

```
# Probabilidades (no etiquetas)
y_prob = pipe.predict_proba(X_test)[:, 1]
```

```
auc = roc_auc_score(y_test, y_prob)
fpr, tpr, _ = roc_curve(y_test, y_prob)
```

```
plt.plot(fpr, tpr, label=f'AUC = {auc:.3f}')
plt.plot([0,1],[0,1], '--', color='gray')
plt.xlabel('FPR'); plt.ylabel('TPR')
plt.legend(); plt.show()
```

AUC $\approx$ 1.0

Clasificador casi perfecto

AUC $\approx$ 0.8

Rendimiento bueno

AUC $\approx$ 0.5

Equivalente a azar aleatorio

 Usa `predict_proba()[0,1]` para obtener probabilidades, no etiquetas. Muchos clasificadores de scikit-learn lo implementan por defecto.

Sesgo y varianza: curvas de validación

El dilema entre sesgo y varianza es central en machine learning. Comprender en qué punto se encuentra nuestro modelo permite tomar decisiones acertadas: más datos, regularización, o un modelo más flexible.



Diagnóstico: underfitting vs overfitting



Alto sesgo (Underfitting)

El modelo es demasiado simple para capturar los patrones. Tanto el error de entrenamiento como el de validación son **elevados**. Solución: modelo más complejo, más características, menos regularización.



Alta varianza (Overfitting)

El modelo memoriza el conjunto de entrenamiento. El error de entrenamiento es bajo pero el de validación es **muy superior**. Solución: más datos, regularización, dropout, o modelo más simple.



Equilibrio óptimo

Mínima diferencia entre error de entrenamiento y validación, ambos en un nivel aceptable. Se alcanza ajustando la capacidad del modelo y la cantidad de datos disponibles.

Curvas de aprendizaje y validación en scikit-learn

Las **curvas de aprendizaje** muestran cómo evolucionan los errores de entrenamiento y validación al aumentar el tamaño del dataset. Las **curvas de validación** hacen lo mismo al variar un hiperparámetro concreto.

```
from sklearn.model_selection import learning_curve, validation_curve
import numpy as np

# Curva de aprendizaje
train_sizes, train_scores, val_scores = learning_curve(
    estimator=pipe, X=X_train, y=y_train,
    train_sizes=np.linspace(0.1, 1.0, 10),
    cv=5, scoring='f1_weighted', n_jobs=-1
)

# Curva de validación (ej. variando max_depth del árbol)
param_range = [1, 3, 5, 7, 10, 15, 20]
train_v, val_v = validation_curve(
    estimator=pipe, X=X_train, y=y_train,
    param_name='model__max_depth',
    param_range=param_range, cv=5
)
```

📌 Nota el prefijo `model__` para referenciar hiperparámetros del paso `model` dentro del Pipeline. Esta sintaxis doble guion bajo (`__`) es estándar en scikit-learn.

GridSearchCV: búsqueda exhaustiva de hiperparámetros

GridSearchCV automatiza la búsqueda del mejor conjunto de hiperparámetros combinando validación cruzada con una exploración sistemática de todas las combinaciones posibles de la cuadrícula definida.



Implementación de GridSearchCV con Pipeline

Al integrar GridSearchCV con un Pipeline, la búsqueda de hiperparámetros abarca tanto el preprocesamiento como el modelo, y la validación cruzada interna garantiza que no haya data leakage en ningún momento.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'model__n_estimators': [50, 100, 200],
    'model__max_depth':    [None, 5, 10],
    'prep__num__strategy': ['mean', 'median']
}

gs = GridSearchCV(
    estimator = pipe_full,
    param_grid = param_grid,
    cv        = 5,
    scoring    = 'f1_weighted',
    n_jobs     = -1,
    verbose    = 1
)

gs.fit(X_train, y_train)

print("Mejores parámetros:", gs.best_params_)
print("Mejor F1 (CV):", gs.best_score_)
y_pred_final = gs.best_estimator_.predict(X_test)
```

RandomizedSearchCV

Cuando el espacio de hiperparámetros es grande, usa RandomizedSearchCV para explorar un subconjunto aleatorio y reducir el tiempo de cómputo significativamente.

Evaluación final

Evalúa `gs.best_estimator_` sobre el test set **sólo una vez** al final. Usar el test set para seleccionar el modelo introduce sesgo de selección.

Elegir la métrica según el negocio

La mejor métrica técnica no siempre es la más relevante para el negocio. Alinear la función de evaluación del modelo con el objetivo real de la organización es uno de los errores más frecuentes —y costosos— en proyectos de ML.



La métrica técnica debe reflejar el coste real

Diferentes contextos de negocio tienen asimetrías distintas entre tipos de error. Elegir la métrica equivocada puede producir un modelo que optimiza algo diferente a lo que realmente importa.



Diagnóstico médico →
Recall alto

Un falso negativo (paciente enfermo clasificado como sano) es mucho más grave que un falso positivo. Maximizar el recall minimiza los casos no detectados. La precisión puede sacrificarse.



Filtro de spam →
Precisión alta

Un falso positivo (correo legítimo marcado como spam) frustra al usuario y puede hacer perder información importante. Priorizar precisión evita bloquear mensajes válidos.



Detección de fraude →
F1 o recall

Cada transacción fraudulenta no detectada tiene un coste directo. Sin embargo, bloquear demasiadas transacciones legítimas daña la experiencia del cliente. F1 equilibra ambos objetivos.

Guía de selección de métricas

Caso de uso	Métrica recomendada	Evitar	Razón
Predicción de precios	RMSE o MAE	R^2 solo	Interpretable en € o \$
Clasificación equilibrada	Accuracy, F1 macro	—	Clases iguales en soporte
Clases desequilibradas	F1 weighted, ROC-AUC	Accuracy	Accuracy es engañosa
Coste asimétrico FN > FP	Recall	Precision	FN muy costoso
Coste asimétrico FP > FN	Precisión	Recall	FP muy costoso
Ranking / probabilidades	ROC-AUC	F1 binario	Umbral-independiente

Resumen del módulo

Este módulo ha cubierto el ciclo completo de trabajo con scikit-learn, desde la API base hasta la optimización de hiperparámetros. Estos conceptos forman la columna vertebral de cualquier proyecto de ML profesional.



API unificada

`fit()`, `transform()`, `predict()` en todos los estimadores



Pipelines

Flujos reproducibles sin data leakage



Cross-validation

Estimación robusta del rendimiento real



Métricas adecuadas

MAE, RMSE, R^2 , precisión, recall, F1, AUC



GridSearchCV

Búsqueda sistemática de hiperparámetros



Orientación al negocio

La métrica debe reflejar el coste real del error



¡A practicar!

El mejor modo de consolidar estos conceptos es aplicarlos sobre un dataset real. Construye un Pipeline completo, evalúa con validación cruzada, analiza las métricas apropiadas para tu problema y ajusta los hiperparámetros con GridSearchCV. Cada paso reforzará tu comprensión.

1

Elige un dataset

Kaggle, UCI ML Repository o los datasets integrados en scikit-learn (`load_breast_cancer()`, `fetch_california_housing()`)

2

Construye el Pipeline

Imputa, escala, codifica y modela en un único objeto encadenado con `ColumnTransformer`

3

Evalúa y diagnostica

Curvas de aprendizaje, matriz de confusión, ROC-AUC. ¿Underfitting u overfitting?

4

Optimiza y decide

GridSearchCV con la métrica que mejor representa el objetivo de negocio de tu problema